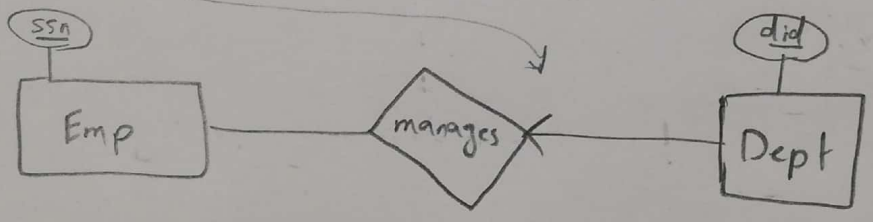
علیرضا کریمی ۸۱۰۱۰۱۴۹۲

۱) الف) data independence یعنی اینکه لول اپلیکیشن و view ما از اپلیکیشن ، از نحوه ذخیره داده و الگوریتم های مربوط به آن abstract باشد و ما متوجه آنها نشویم (متوجه نشیم که توی data base چه اتفاقی می‌افته)

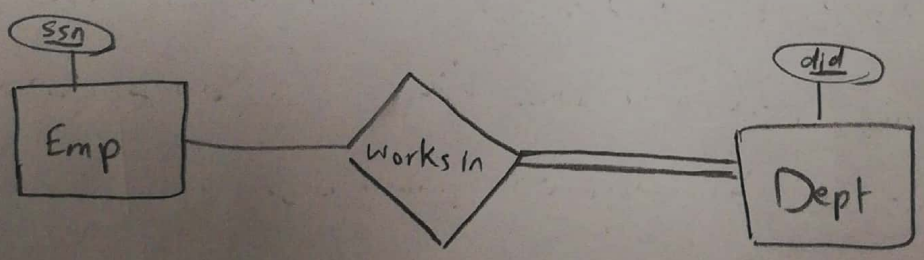
logical data indep... یعنی اینکه اگر تغییری در داده ها ایجاد شد (داده ها یا جدول های جدید ایجاد و یا آپدیت شد) ، در سطح view های مختلف از اپلیکیشن ، ما متوجه تغییرات نشویم و به شکلی برخوردیم . دید ما از تغییرات داده ها ، abstract باشد .

physical data indep... یعنی اینکه اگر نحوه ذخیره سخت‌افزاری و عملکرد تجزیه و تحلیل داده ها متفاوت شد ، دید ما (view / فرمت) از این تغییرات abstract باشد . مثلا شاید ما query هایی نوشته باشیم (مثل دیتابیت sort شده داریم) و از طرفی الگوریتم sort رو سریع تر کنیم در پیاده سازی به اون query ها ، endUser از این تغییر ، تاثیر منفی نگیرد و مشکلی در کارش ایجاد نشود و abstract از این قضیه باشد.
(مثلا تغییر در پیاده سازی الگوریتم ها ، حسب وجه ، sort و ...)

ب) Key constraint : وقتی که محدودیت حداکثر بدونه رو در رابطه‌مون داریم . مثلا وقتی می خوایم بگیم که هر department ، حداکثر یک مدیر (از میان کارمندان) دارد . و به این شکل نشان می دهیم (بالشی) :

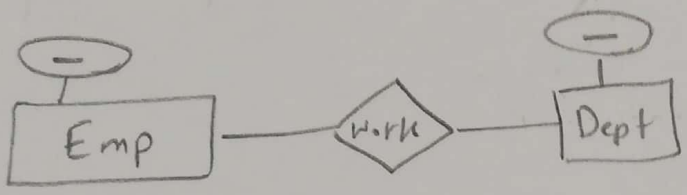


participation constraint : وقتی می خوایم بگیم حداقل بدونه از اون entity را داریم در رابطه . مثلا وقتی می خوایم بگیم که هر department ، حداقل ۱ کارمند دارد که در آن کار می‌کند ، به این شکل نشان می دهیم (یک خط تودیر یا دوخط موازی)

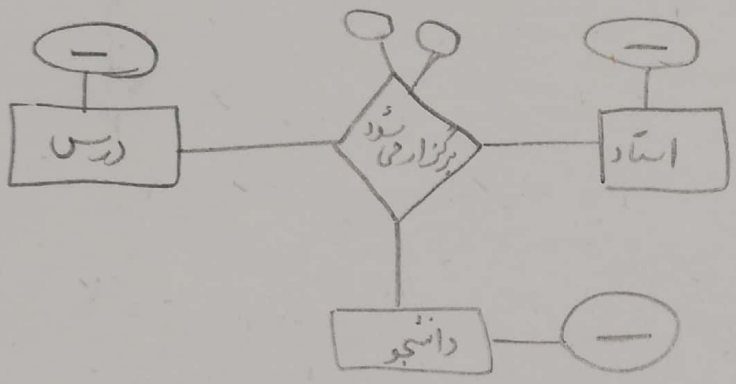


ادامه ۱)

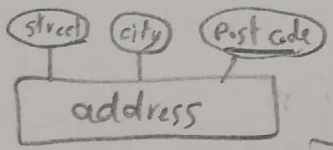
ج) در رابطه باینری، دوتا entity با هم در یک relation هستند. مثلا می خواهیم بگیم کدوم کارمند(ها) در کدوم ساختمان(ها) کار می کند. پس داریم:



در رابطه ترنری، ۳ تا entity با هم در یک رابطه هستند. مثلا می خواهیم بگیم کدام درس(ها) با کدام استاد(ها) و کدام دانشجو(ها) برگزاری شود. پس داریم:

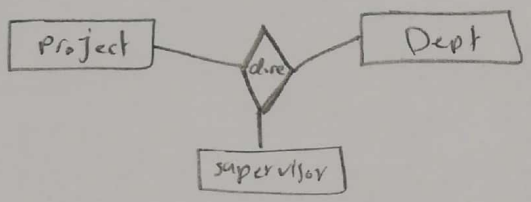


۲) به طور مثال فرض کنید که می خواهیم برای address تصمیم گیری کنیم که آیا attribute یک نفر باشد یا یک entity جدا باشد؟ اگر بخواهیم تجزیه و تحلیلی روی آن انجام بدهیم (در این مثال، مثلا اگر بخواهیم که تحلیلی روی آدرس و بخش های مملکت آن انجام دهیم، مثلا بخواهیم همه افرادی را پیدا کنیم که آدرس خانه آنها در خیابان نلان هست)، در این صورت، آن را به صورت یک entity در نظر می گیریم و مثلا داریم

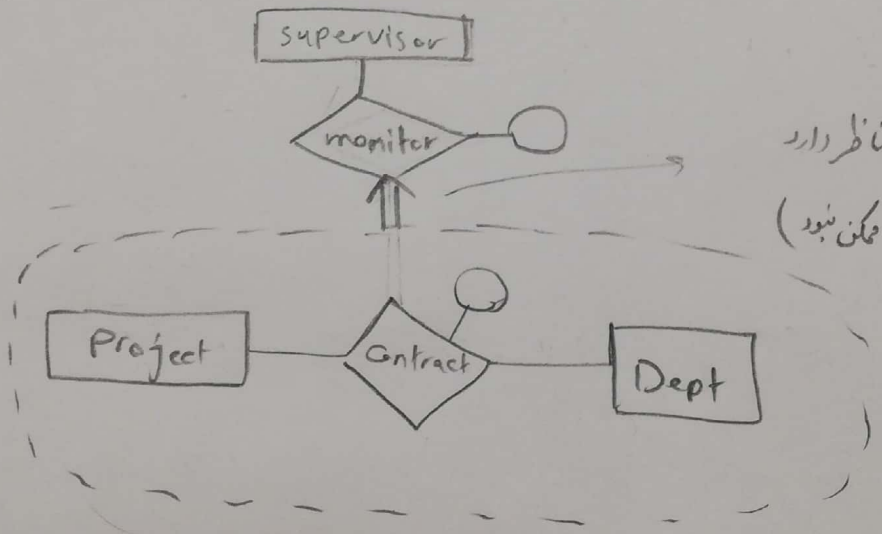


اما اگر تنها دید atomic به آن ویژگی (آدرس) داریم و نیازی به تجزیه و تحلیل آن نیست و تنها یک string آدرس برای ما کافی است، آن را به صورت attribute می گیریم (در این حالت اگر entity بگیریم هزینه خیلی بیشتری می شه) در صورتی که بخواهیم چند تا از این ویژگی (آدرس) برای هر فرد داشته باشیم، باید آن ویژگی (آدرس) را entity کنیم. در صورتی که اطلاعات مربوط به آن ویژگی آدرس، ارتباط چندانی با entity های دیگر مثل entity هر فرد ندارد، بهتر است برای آدرس یک entity جدا و مستقل در نظر بگیریم؛ چون اطلاعات آن به entity های دیگر زیاد مربوط نیست و آنها را به طور نامربوطی شلوغ می کند.

۳) اگر به دنبال نمایش ساده و ذخیره سازی اطلاعات به صورت ساده هستیم و محدودیت ارث خاصی هم نداریم و اطلاعات خاص متفاوتی مربوط به فقط کار نظارت یا فقط کار قرارداد نداریم، همان رابطه ternary کافی است.

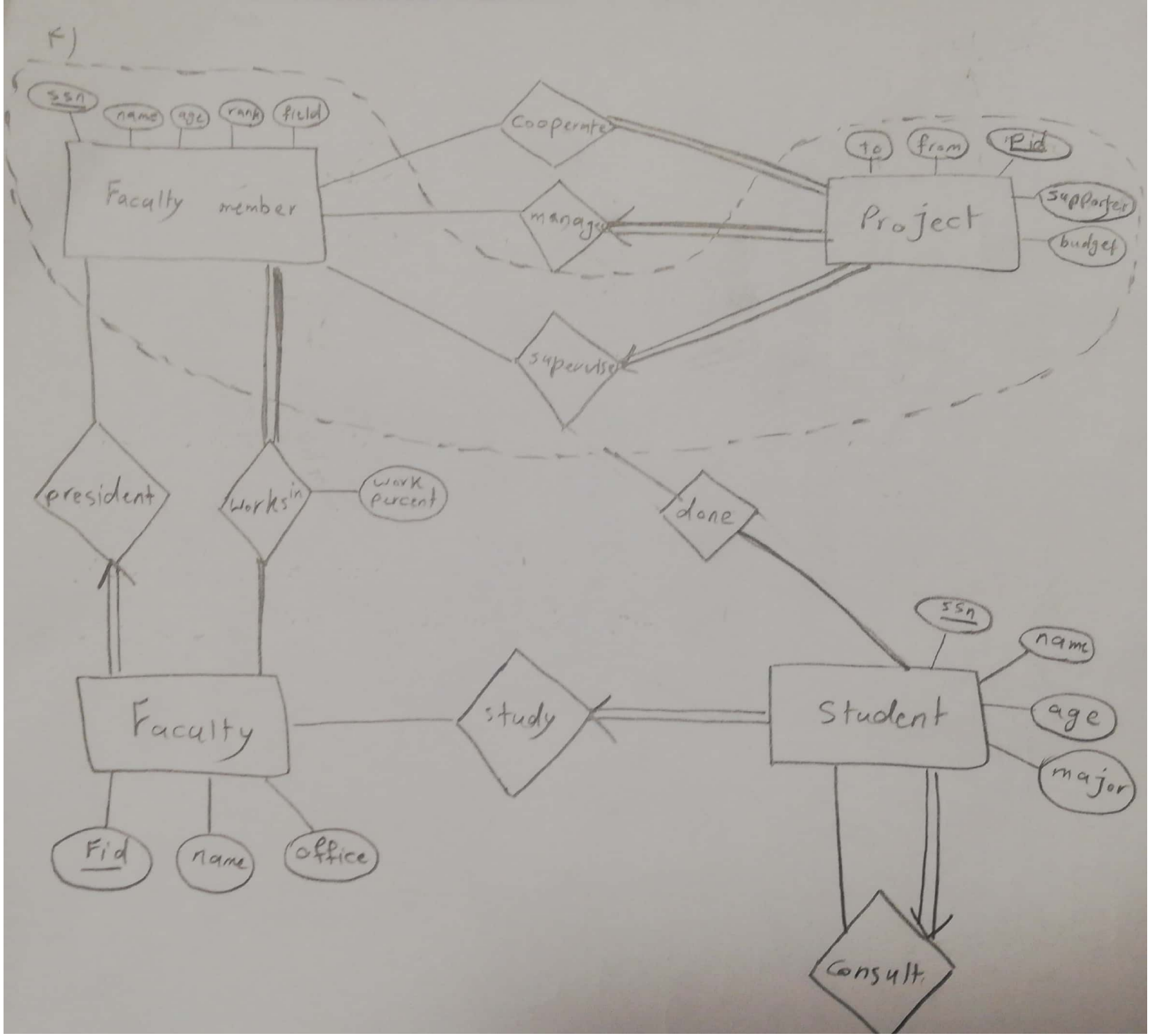


اما اگر به عنوان مثال، یک attribute ای خاص برای relation قرارداد داریم و یک attribute متفاوت دیگری برای relation نظارت کردن داریم و این ها به هم مربوط نیستند و می خواهیم این تمایز را نشان دهیم، از رابطه aggregation استفاده می کنیم. در صورتی هم که نخواهیم بگوییم هر قرارداد حداکثر و یا حداقل توسط یک ناظر انجام شود، تنها به کمک aggregation می توانیم آن را پیاده سازی کنیم و این امکانات در رابطه ternary وجود ندارد. و حتی پیچیدگی طراحی aggregation هم بیشتر است اما امکانات آن هم (ایجاد محدودیت ارد دقیق تر رابطه) بیشتر است



هر قرارداد تنها یک ناظر دارد
(این کار در رابطه ternary ممکن نبود)

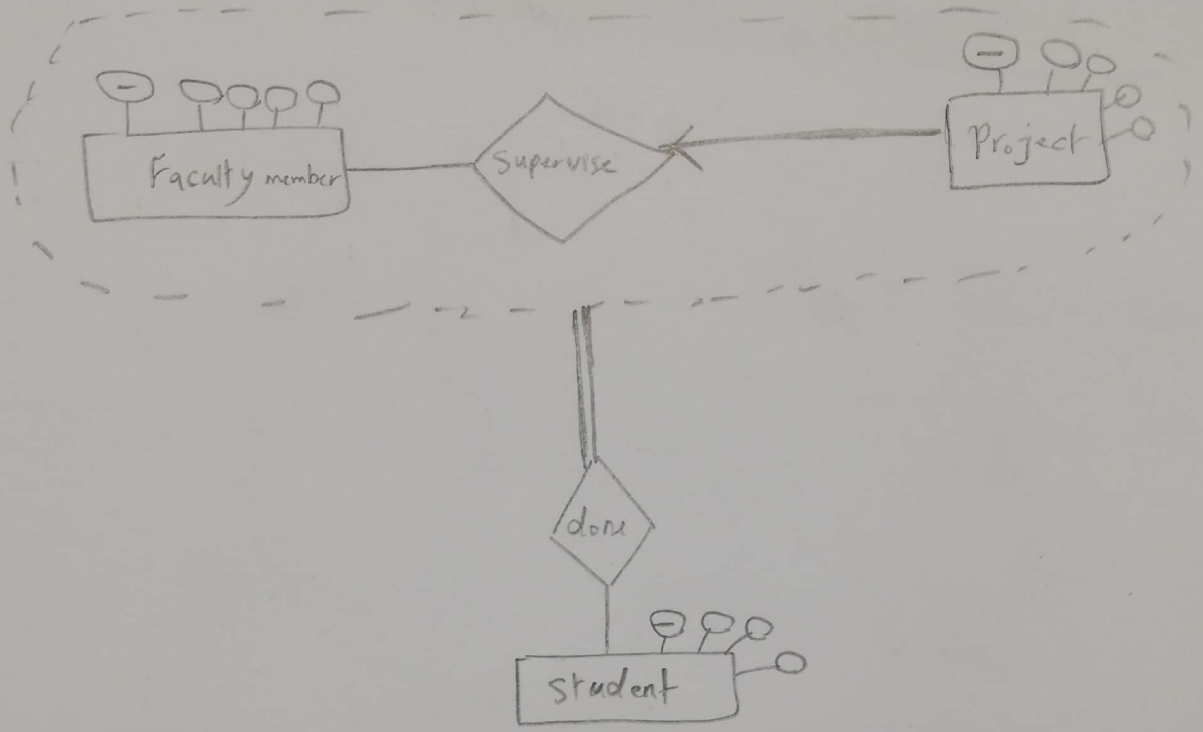
علاوه بر موارد بالا، رابطه بین Dept و project می تواند به هر گونه باشد و این موضوع از عملیات monitor کردن توسط ناظر، مستقل و abstract است. (منظورم اینه که هر Dept می تونیم محدودیت بذاریم که حداقل یا حداکثر ۱ پروژه بگیره یا می تونیم محدودیت بذاریم که هر پروژه حداقل یک یا حداکثر یک ۱ دپارتمان باشه و این موضوع روی موضوع نظارت هیچ تاثیری نمی گذاره. در صورتی که این امکان در رابطه ternary امکان پذیر نبود


F)
SSN
name
age
rank
field
Faculty member
Coopernte
manage
supervise
to
from
Rid
Project
supporter
budget
president
Works in
work percent
done
SSN
name
age
major
Student
study
Faculty
Fid
name
office
Consult

ادامه ۴)

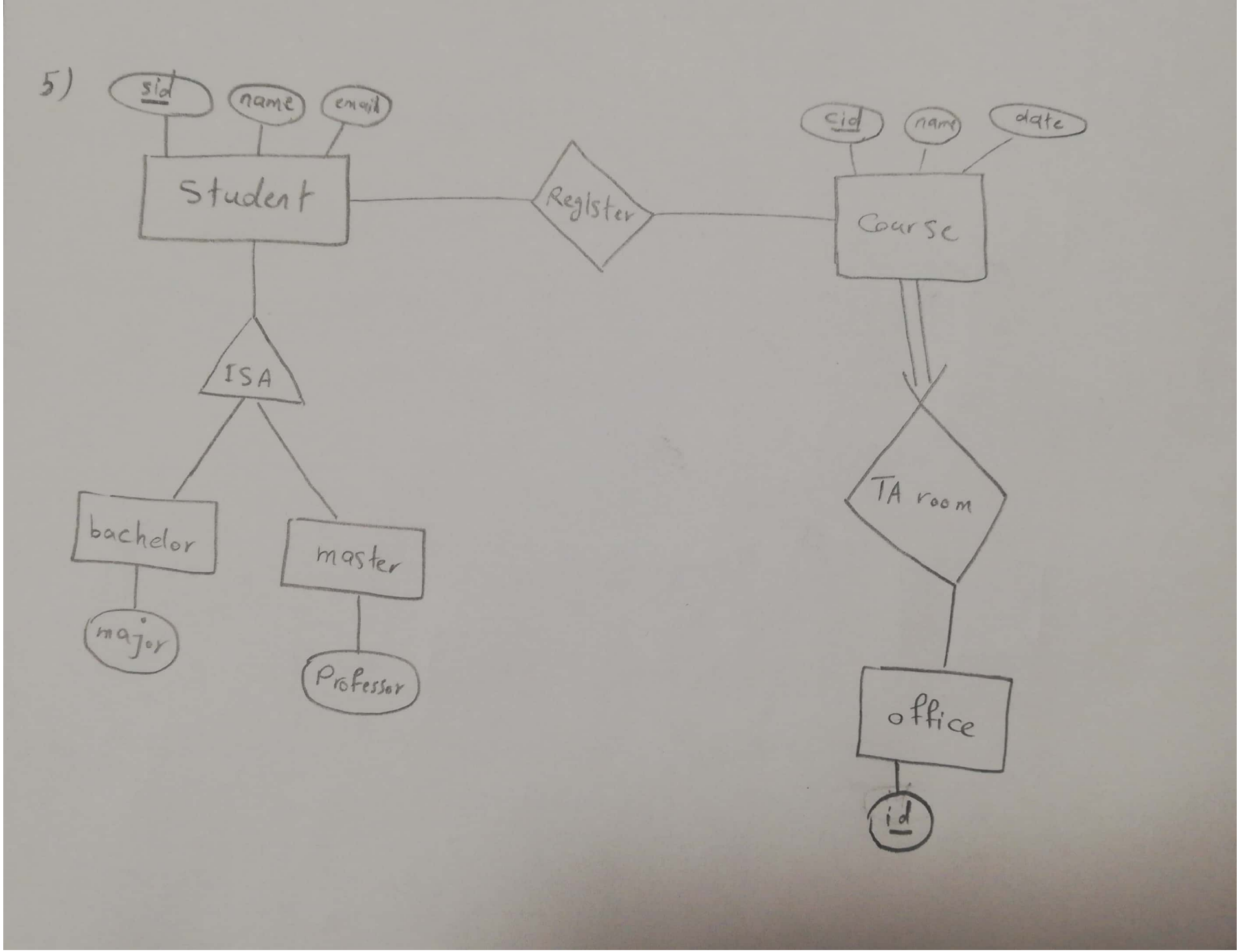
نمایش ER گفته شده، این فرض را در نظر می‌گیرد که هر پروژه، دقیقاً یک ناظر استاد دارد. چه دانشجو باشد و چه دانشجو نباشد.

در صورتی که اگر هر پروژه، زمانی ناظر استاد داشته باشد که حداقل یک دانشجو داشته باشیم، باید این بخش مربوط به عمل نظارت رو به این شکل در نظر گرفت:



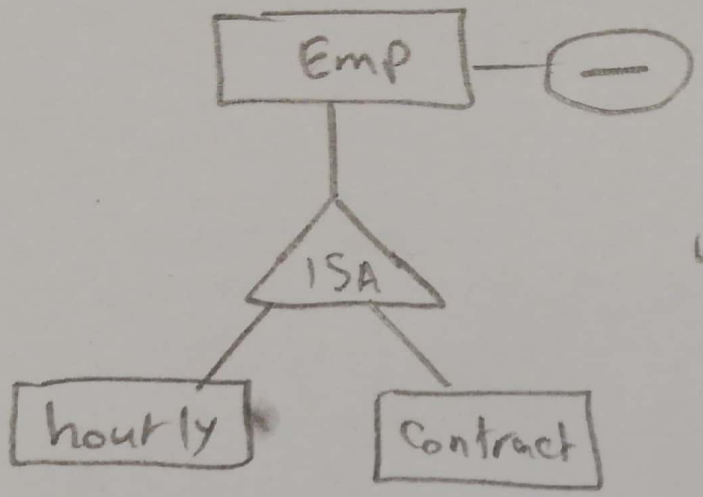
این یعنی هر پروژه حداکثر یک ناظر دارد، در صورتی که ناظر داشته باشد، حداقل باید یک دانشجو هم داشته باشد و در صورتی که ناظر نداشته باشد، اصلا در رابطه نخواهد آمد و در رابطه قرار نمی‌گیرد (و دانشجویی هم ندارد) یک جوری force کردیم که اگر دانشجو دارد، حتما استاد ناظر هم داشته و در غیر این حالت، در رابطه قرار نمی‌گیرد

5)



CamScanner

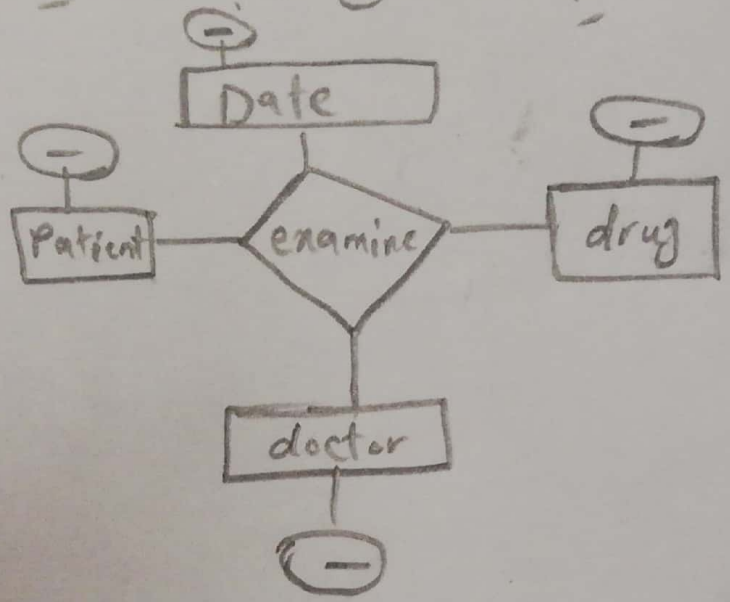
۶) فرض کنید که داشته باشیم:



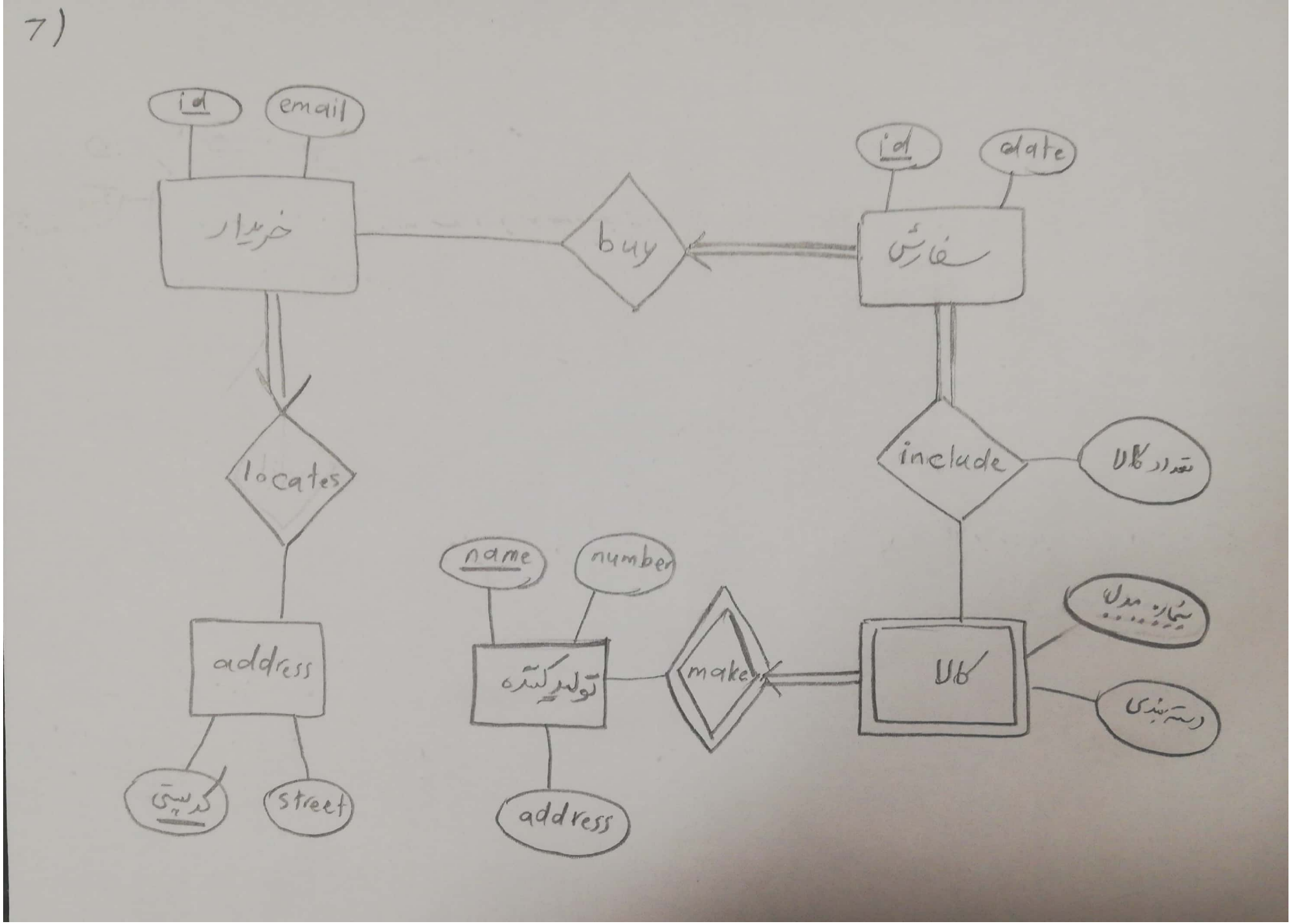
overlap constraint : یعنی اینکه آیا می‌تونه یک چیز در چند تا زیرمجموعه بیاد؟ مثلا آیا یک کارمند هم می‌تونه قرارداد ساعتی باشه و هم ساعتی؟
(یعنی آیا ما مجاز به انجام این کار هستیم یا نه که یک کارمند رو در چند زیرمجموعه قرار بدیم؟)

covering constraint : یعنی اینکه مثلا یک چیزی در هیچ کدام از اون زیرمجموعه‌ها نباشه. یعنی اینکه مثلا یک کارمند، نه قرارداد ساعتی باشه و نه ساعتی. (یعنی اینجا این سوال مطرحه که آیا همه کارمندا در همه زیرمجموعه‌ها قرار می‌گیرن یا کارمندی وجود داره که در هیچ کدوم نباشه؟)

ادامه ۶) حتی کامل شاید درست نباشه. مثلا اگر یک دکتر برای یک بیمار، یک آزمایش رو در تاریخ های مختلف تجویز کنه، فقط یکی از آنها (مثلا آخری) ذخیره می‌شود. برای این کار باید یک entity اضافه کنیم که در آن تاریخ ها را به صورت یکتا نگه می داریم و آنگونه می‌توانیم تجویز یک بیمار در تاریخ های مختلف داشته باشیم



7)


id
email
خریدار
buy
id
date
سفارش
locates
include
تعداد کالا
address
name
number
تولیدکننده
make
کالا
سایز، مدل
دسته‌بندی
کدپستی
street
address